

Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0119 P.I. de Arquitectura y Ensamblador

Proyecto Integrador de Arquitectura y Ensamblador

Cuarto Avance

Número de equipo: 2

Profesor: Willian Baltodano Cubillo

Integrantes:

- Melissa Garita Chacón C23186

-Andrés Ramírez Solís C36467

- Jimena Bejarano C31074

-Stephanie Monge C35035

I. Objetivos

Optimización con lenguaje ensamblador

El lenguaje ensamblador permite acceder directamente a los registros internos del microcontrolador y a las instrucciones de bajo nivel del hardware, lo que proporciona un control mucho más preciso y eficiente que el que ofrece el lenguaje C.

En C, cada operación (como una lectura analógica o una comparación) pasa por varias capas de abstracción del compilador y las librerías, mientras que en ensamblador se puede reducir el número de instrucciones ejecutadas, aprovechar instrucciones específicas del microcontrolador (como IN, OUT, MOV, ADC, SBI, CBI, entre otras) y minimizar los ciclos de reloj empleados por cada lectura o escritura.

En este proyecto, que involucra sensores de sonido y respuesta inmediata mediante LEDs o ventilador, el tiempo de respuesta del sistema es fundamental. Usar ensamblador permite eliminar retardos innecesarios y hacer que la detección del sonido se ejecute en menos ciclos de CPU, logrando que los LEDS respondan casi en tiempo real.

En resumen, el lenguaje ensamblador se utiliza para optimizar tareas que requieren velocidad o control fino del hardware, como la lectura analógica rápida, el control directo de pines digitales o el manejo eficiente de interrupciones.

Código en C crítico

Durante el análisis del código, se identificó que la función `medirNivel()` es la parte más crítica en cuanto a tiempo de ejecución.

Esta función realiza un bucle de múltiples lecturas analógicas (`analogRead()`), buscando los valores máximos y mínimos para calcular la diferencia de amplitud del sonido captado por los sensores.

Cada llamada a `analogRead()` implica varios pasos internos:

- Seleccionar el canal analógico del ADC.
- Esperar el tiempo de conversión (aproximadamente 100 μ s por lectura).
- Leer el valor resultante de los registros del conversor analógico-digital (ADC).

Este proceso se repite muchas veces (100 lecturas por sensor), lo que consume una cantidad significativa de tiempo de CPU y retrasa la actualización del sistema.

Por esta razón, es el punto ideal para reemplazar con una rutina en ensamblador inline, donde se pueden controlar directamente los registros del ADC (por ejemplo, `ADMUX`, `ADCSRA`, `ADCL`, `ADCH`) y realizar la lectura más rápida y eficiente, reduciendo el tiempo total del ciclo de muestreo.

El ADC (Analog-to-Digital Converter) es el módulo del microcontrolador encargado de convertir una señal analógica (como la del sensor de sonido) en un valor digital que el programa pueda interpretar.

En el Arduino Uno (basado en el microcontrolador ATmega328P), este conversor es de 10 bits, por lo que devuelve valores entre 0 y 1023, dependiendo del voltaje de entrada (0V a 5V).

El funcionamiento del ADC se controla a través de varios registros especiales. Los principales son los siguientes:

- *ADMUX – ADC Multiplexer Selection Register: Define qué entrada analógica se desea leer, por ejemplo A1, A2, etc.. y cuál va a ser la referencia de voltaje usada por conversor.*
- *ADCSRA – ADC Control and Status Register A: Controla el inicio de la conversión, la habilitación del ADC, el modo automático y la velocidad del reloj del ADC*
- *ADCL y ADCH – ADC Data Register Low y ADC Data Register High: Se almacena el resultado de la conversión analógica-digital. Como el ADC es de 10 bits, el resultado se reparte 8 LSB en ADCL y 2 MSB en ADCH*

II. Selección del módulo o los módulos

Como se estableció en el apartado anterior, el módulo seleccionado para optimizar es el proceso de lectura de los sensores de sonido. Esta sección del código es la que se ejecuta con mayor frecuencia dentro del ciclo principal (loop()), ya que en cada iteración el sistema realiza múltiples lecturas analógicas para calcular el nivel de ruido detectado.

El tiempo de respuesta en esta etapa es crítico, porque de ella depende que los LEDs o el ventilador reaccionen de forma inmediata ante un estímulo sonoro.

Actualmente, esta función utiliza la instrucción `analogRead()`, la cual, aunque es sencilla de usar, introduce una latencia considerable debido a las múltiples operaciones que realiza internamente en C (configuración del ADC, inicio de conversión, espera de finalización y lectura del resultado).

Por esta razón, se propone reemplazar esta parte por una versión implementada en lenguaje ensamblador (inline assembly), que permita acceder directamente a los registros del ADC (ADMUX, ADCSRA, ADCL, ADCH), logrando así una conversión más rápida y precisa sin pasar por las capas de abstracción de la biblioteca estándar.

III. Analizar el comportamiento actual del módulo

Actualmente, el módulo de lectura está implementado en la función `medirNivel(int pin)`, escrita en C dentro del entorno de Arduino. Esta función realiza un ciclo de lecturas sobre el pin analógico especificado, utilizando `analogRead()` para obtener valores del sensor de sonido.

Durante las pruebas prácticas, se observó que el sistema solo responde cuando se sopla directamente al sensor, pero no detecta adecuadamente los aplausos, que eran el estímulo esperado originalmente. Además, la respuesta no es constante: a veces reacciona al soplido y otras no, lo que indica una baja sensibilidad o lentitud en la detección.

Este comportamiento se debe a que la lectura mediante `analogRead()` introduce un retardo aproximado de 100 microsegundos por conversión, y al realizarse decenas o cientos de lecturas por ciclo (en este caso 150 por sensor), el tiempo total de muestreo se vuelve demasiado alto.

Como consecuencia, el sistema no puede captar correctamente los picos rápidos de presión sonora que produce un aplauso, ya que estos duran solo unos pocos milisegundos.

Por tanto, se concluye que este módulo es el más crítico en tiempo y rendimiento dentro del sistema general, y que optimizarlo mediante acceso directo al hardware ADC desde ensamblador permitirá una lectura más veloz, confiable y precisa, mejorando significativamente la respuesta del proyecto frente a estímulos sonoros.

Tiempo de Ejecución

Para analizar el tiempo de ejecución se añadieron mediciones con las funciones `micros()` para garantizar una medición más exacta y así poder analizar el tiempo promedio de ejecución del módulo `medirNivel(int pin)` y `millis()` para analizar la totalidad del tiempo de

ejecución del loop. En las pruebas obtenidas se registró un tiempo promedio de 11,2 ms por cada ejecución de `medirNivel(int pin)`. Mientras que el tiempo total del loop fue de 31ms. Dado que esta función dentro del código se ejecuta 2 veces, para el sensor1 y el sensor2, se puede concluir que esta representa aproximadamente 22 ms del tiempo total del loop, lo que quiere decir que, representa un estimado del 72% del tiempo total del procesamiento. Por lo que, se puede afirmar de acuerdo a los datos obtenidos, `medirNivel(int pin)` es el componente más demandante del sistema y el más apto para optimizarse.

IV. Propuesta de optimización

En el código, la función *medirNivel()* se encarga de realizar las lecturas analógicas mediante un *loop*, esto puede generar retrasos en la detección del aplauso por el tiempo de procesamiento consumido debido a la naturaleza de los bucles en C++, los cuales a pesar de eficientes, se puede mejorar su tiempo de respuesta.

Para crear esta optimización, se reescribe la implementación en C++ de esta función a Inline Assembly. Esto permite manejar de manera directa los registros y ajustar cada instrucción para aprovechar al máximo la CPU, lo cual se traduce en una velocidad de la detección del sonido mejorada de milisegundos a microsegundos, logrando un sistema más rápido y sensible sin modificar la estructura general del programa.

Al reescribir parte del código en lenguaje ensamblador AVR, se busca optimizar las secciones críticas del programa, especialmente aquellas relacionadas con la conversión analógica-digital (ADC) y el procesamiento inmediato de las lecturas. Estas operaciones requieren un control preciso del hardware y una respuesta rápida, por lo que el uso de ensamblador permite una manipulación directa de los registros del microcontrolador sin depender de las capas de abstracción del compilador en C.

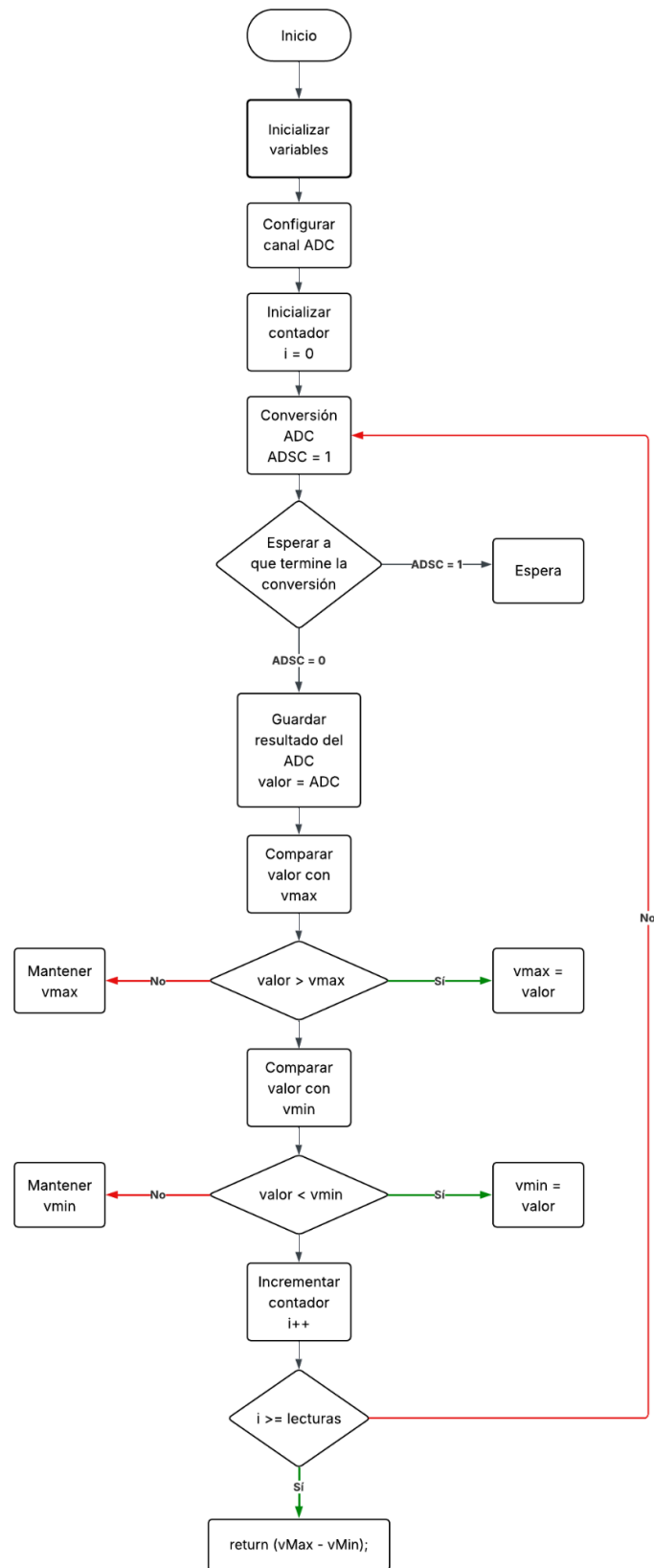
En particular, se espera una mejora notable en el tiempo de ejecución, al reducir la cantidad de instrucciones generadas y eliminar ciclos de reloj innecesarios. Además, el acceso directo a los registros como ADMUX, ADCSRA, ADCL y ADCH permite un control más fino sobre los bits de configuración y resultados, optimizando tanto la selección del canal como el ajuste de la referencia de voltaje. Por último, el uso de ensamblador promueve un uso más eficiente de los registros del CPU, evitando variables temporales en memoria y aprovechando mejor los recursos internos del microcontrolador.

V. Implementación Híbrida

Para esta cuarta etapa, se realizó una implementación híbrida en el código, como se ha mencionado anteriormente en este reporte, para reemplazar el módulo `medirNivel(int pin)` por lenguaje ensamblador AVR. En la versión optimizada, se implementó código en ensamblador embebido (`inline assembly`) dentro de la misma función `medirNivel(int pin)`. Este código controla directamente los registros del ADC (`ADMUX`, `ADCSRA`, `ADCL` y `ADCH`). Esto permitió eliminar la sobrecarga de la función `analogRead()` y manejar manualmente el ADC del ATmega328P, iniciando la conversión con la instrucción `sbi`, esperando su finalización con `sbic` y leyendo el valor crudo en 10 bits directamente desde `ADCL` y `ADCH`. El resto de la lógica del sistema (control de LEDs, lectura del botón y actualización del LCD) permanece en C, obteniéndose así una integración mixta eficiente.

Para analizar el impacto de la optimización, se añadieron mediciones utilizando `micros()` para obtener el tiempo exacto de ejecución de la función `medirNivel(int pin)` y `millis()` para evaluar la duración total del loop. En la versión original escrita completamente en C, se registró un tiempo promedio de 11,2 ms por cada ejecución de `medirNivel(int pin)`, lo que representaba aproximadamente 22 ms por ciclo, ya que la función se llamaba dos veces, además de un tiempo total de loop cercano a 31 ms. Tras implementar la optimización mediante ensamblador embebido, las pruebas mostraron una reducción drástica del tiempo de ejecución: la función ahora tarda en promedio 112–116 μ s por sensor, y el tiempo total del loop disminuyó a aproximadamente 8–9 ms. Esto representa una mejora de más del 98% de reducción en el tiempo del módulo crítico y una disminución cercana al 70% en el tiempo total del loop. Con base en estos resultados, se confirma que la optimización no solo fue exitosa, sino que transforma `medirNivel(int pin)` de ser el componente más costoso del sistema a uno altamente eficiente, mejorando significativamente el desempeño global de la aplicación.

Diagrama del módulo



VI. Referencias

- AVR Tutorials - Structure of an Assembly Language Program.* (s. f.). https://www-rjhcoding-com.translate.goog/avr-asm-structure-of-an-asm-program.php?_x_tr_sch=http&_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
- AVR Assembler.* (s. f.). <https://ww1.microchip.com/downloads/en/DeviceDoc/40001917A.pdf>
- ADC Programming Registers.* (2019, 19 febrero). Arduino Forum. <https://forum.arduino.cc/t/adc-programming-registers/575105>
- inline ASM on Arduino UNO.* (2018, 17 julio). Arduino Forum. <https://forum.arduino.cc/t/inline-asm-on-arduino-uno/536979>
- Engineer, C. (2021, 4 julio). *ADC Register Description : Arduino / ATmega328p*. Arnab Kumar Das. https://www-arnabkumardas-com.translate.goog/arduino-tutorial/adc-register-description/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc